Group A: Arrays and Searching Sorting Algorithms

Experiment 2

Title: Employee Salary Sorting and Top Five Highest Salary Identification

Objectives:

- To implement Selection Sort and Bubble Sort algorithms in Python.
- To sort a list of employee salaries in ascending order using both algorithms.
- To identify and display the top five highest salaries from the sorted list.

Problem Statement:

Given a list of employee salaries as floating-point numbers, sort them in ascending order using Selection Sort and Bubble Sort algorithms, and then extract and display the top five highest salaries.

Outcomes:

- A Python program demonstrating the implementation of Selection Sort and Bubble Sort.
- Sorted lists of employee salaries using both algorithms.
- Identification and display of the top five highest salaries.

Software and Hardware Requirements:

- **Software:** Python 3.x interpreter.
- **Hardware:** A computer capable of running Python programs.

Date of Completion: October 6, 2025

Assessment grade/marks and assessor's sign: (To be filled by assessor)

Theory Concept in brief:

- **Selection Sort:**

  This algorithm repeatedly finds the minimum element from the unsorted part of the list and puts it at the beginning. It maintains two subarrays in a given array: one already sorted and one remaining unsorted.

- **Bubble Sort:**

  This algorithm repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. The pass through the list is repeated until no swaps are needed, indicating that the list is sorted.

Algorithm:

1. Selection Sort:

- Initialize n as the length of the salary list.
- Iterate from i = 0 to n-1:
  a. Set min_idx = i.
  b. Iterate from j = i+1 to n:
  i. If salaries[j] < salaries[min_idx], set min_idx = j.
  c. Swap salaries[i] and salaries[min_idx].

2. Bubble Sort:

- Initialize n as the length of the salary list.
- Iterate from i = 0 to n-1:
  a. Set swapped = False.
  b. Iterate from j = 0 to n-i-1:
  i. If salaries[j] >salaries[j+1], swap salaries[j] and salaries[j+1].
  ii. Set swapped = True.
  c. If swapped is False, break the outer loop (list is sorted).

3. Display Top Five:

- Reverse the sorted list to get descending order.
- Print the first five elements of the reversed list.
  Flowchart: (Cannot be directly represented in text, but would depict the sequential steps of each sorting algorithm and the final display of top salaries.)

Test Cases:

Test Case 1:

- **Input:** salaries = [1500.0, 2500.5, 1200.75, 3000.0, 1800.25, 2800.0, 3500.0, 2000.0]
- **Expected Output (Sorted):** [1200.75, 1500.0, 1800.25, 2000.0, 2500.5, 2800.0, 3000.0, 3500.0]
- **Expected Output (Top 5):** [3500.0, 3000.0, 2800.0, 2500.5, 2000.0]
  Test Data Set: (See Test Case 1)

Mathematical Model: (Not applicable for this problem, as it focuses on algorithmic implementation rather than mathematical modeling.)

Conclusion/Analysis:

This program successfully demonstrates the implementation of Selection Sort and Bubble Sort algorithms for sorting employee salaries. Both algorithms effectively sort the list in ascending order. After sorting, extracting the top five highest salaries is achieved by reversing the sorted list and accessing the first five elements. The choice between Selection Sort and Bubble Sort depends on factors like data size and performance requirements, with more efficient algorithms typically preferred for larger datasets.

Program:-

```
defselection_sort(arr):
n = len(arr)
foriin range(n):
min_idx = i
forjin range(i + 1, n):
ifarr[j] <arr[min_idx]:
min_idx = j
arr[i], arr[min_idx] = arr[min_idx], arr[i]
returnarr

defbubble_sort(arr):
n = len(arr)
foriin range(n - 1):
swapped = False
forjin range(0, n - i - 1):
ifarr[j] >arr[j + 1]:
arr[j], arr[j + 1] = arr[j + 1], arr[j]
swapped = True
ifnot swapped:
break
returnarr

# Employee salaries
employee_salaries = [1500.0, 2500.5, 1200.75, 3000.0, 1800.25, 2800.0, 3500.0,
2000.0]

# Sort using Selection Sort
salaries_selection_sorted = selection_sort(list(employee_salaries))
print("Salaries sorted with Selection Sort:", salaries_selection_sorted)

# Sort using Bubble Sort
salaries_bubble_sorted = bubble_sort(list(employee_salaries))
print("Salaries sorted with Bubble Sort:", salaries_bubble_sorted)

# Display top five highest salaries (from either sorted list, they will be
the same)
top_five_salaries = sorted(employee_salaries, reverse=True)[:5]
print("Top five highest salaries:", top_five_salaries)
```